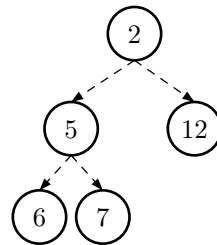
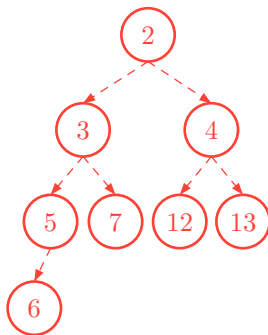


1 Heaperations

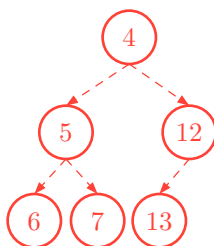
Consider the following min-heap:



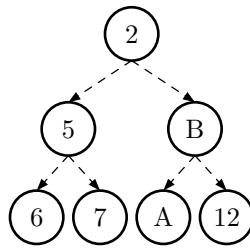
- (a) Draw the heap after inserting 4, 13, and 3 in that order.



- (b) Now, draw the heap after 2 calls of **removeMin**.



- (c) Suppose we recover the original heap and insert two elements of unknown value, A and B , **in that order**.



For each of the following conditional statements, write whether they are **always** true, **sometimes** true, or **never** true, and briefly explain why.

$A < B$

Never. The min-heap property states that a node's children must be greater than or equal to itself. $A \geq B$ implies that A cannot be less than B .

$B > 5$

Sometimes. Without any other information, a node's position relative to other nodes at the same depth does not give us any information about the node's value.

$A \geq 12$

Always. If A were less than 12, then it would have bubbled up to a depth of 1, with 12 as a left child. There are two cases for what happens next:

- If B is less than A , then B would swap with A , but 12 would remain as a left child. This is not the case.
- If B is greater than or equal to A , then A would remain at depth 1. This is not the case in the given heap.

- (d) Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are distinct.

removeMin has a best-case runtime of $\Theta(1)$ and a worst-case runtime of $\Theta(\log N)$

insert has a best-case runtime of $\Theta(1)$ and a worst-case runtime of $\Theta(\log N)$

A **DFS preorder** or **BFS** traversal on a min-heap may output the elements in sorted order. (Assume there are at least 3 elements in the min-heap)

The smallest item of a min heap is at the top, so whatever traversal we choose must output the top element first in a complete binary tree. Only preorder and level-order have this property.

The fourth smallest element in a min-heap with 1000 distinct elements can appear in 14 different places in the heap.

The 4th smallest item can be on the 2nd, 3rd, or 4th level of the heap (1-indexed)

Given a min-heap with 2^{N-1} **distinct** elements, for an element

- to be on the second level it must be less than $2^{N-1} - 2$ element(s) and greater than 1 element(s).
- to be on the bottommost level it must be less than 0 element(s) and greater than $N - 1$ element(s).

An element on the second level must be larger than the root and less than the elements in its subtree. There are $2^{N-1} - 2$ elements in the subtree of an element on the second level: half the elements in the tree minus the root, then subtracting off the node itself.

An element on the bottom level must be greater than all elements on the path from itself to the root. A min heap with 2^{N-1} elements has N levels, so there are $N - 1$ items above it on a path to the root.

Hint: A complete binary tree (with a full last-level) has $2^N - 1$ elements, with N being the number of levels.

2 Heaps, the Hard Way

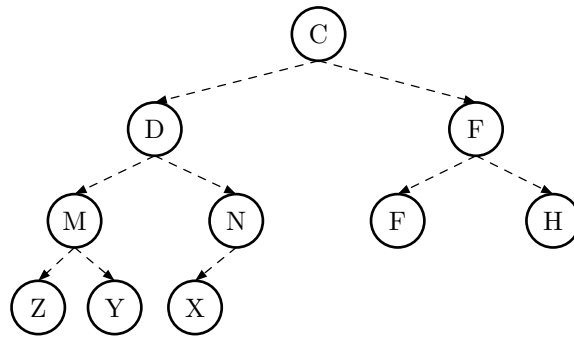
In lecture, we implemented a heap using a backing array. We could also have done so with a tree representation, given below:

```
private class HeapNode {
    private T item;
    private HeapNode left;
    private HeapNode right;

    HeapNode(T i) {
        item = i;
    }
}
```

One issue with this approach is that it is somewhat tricky to find where to insert new nodes. In this problem, we'll walk through this in some detail.

- (a) Suppose we have a heap that currently looks like the figure below, where the integer beside each node is the “node number”, i.e. position in the heap, of each item.



If we add a new item, which node will be the parent of the new node (before doing any **bubbleUp** operations)? Why?

To satisfy the completeness property, N must be the parent of the new node.

- (b) Fill in the `isRightChild(int i)` and `parent(int i)` methods. Assume for `isRightChild` that `i` is nonnegative and for `parent` that `i` is positive. Notice that the root has node number zero.

- `isRightChild` returns true if the node number `i` corresponds to a right child. E.g. `isRightChild(2)` and `isRightChild(4)` should evaluate to `true`, and `isRightChild(0)` and `isRightChild(1)` should evaluate to `false`
- `parent` returns the node number of the parent of the given node `i`. E.g. `parent(9)` should evaluate to 4

```
private boolean isRightChild(int i) {
    return i % 2 == 0 && i > 0;;
}

private int parent(int i) {
    return (i - 1) / 2;
}
```

- (c) Fill in the `followPath(List path)` method so that it matches the given javadoc comment. Assume the given path is valid.

```
/** Return the HeapNode at the end of the given path.
 * For example if path is: false, true, false
 * We will go left, right, left, landing at node #9 */
```

```
private HeapNode followPath(List<Boolean> path) {
    HeapNode p = root;

    for (boolean b : path){
        if (b) {
            p = p.right;
        } else {
            p = p.left;
        }
    }
    return p;
}
```

- (d) The method below will return the i^{th} node. For example, if we call `getNode(9)` on the example heap, we will get the `HeapNode` containing X.

```
private HeapNode getNodeI(int i) {
    List<Boolean> pathToI = constructPath(i);
    HeapNode nodeNumberI = followPath(pathToI);
    return nodeNumberI;
}
```

Fill in the `constructPath` method so that the `getNodeI` method above works correctly. You may use any method from the previous parts of this problem. *Hint: Recall that `LinkedList` has methods `addFirst`, `addLast`, `removeFirst`, and `removeLast`.*

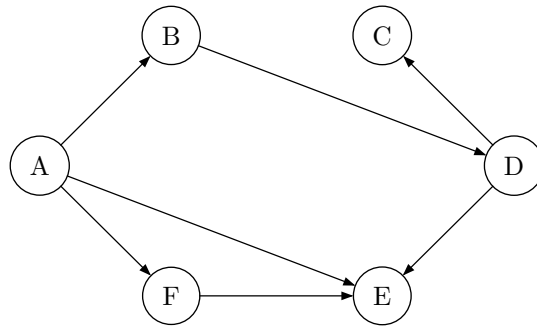
```
private List<Boolean> constructPath(int i) {
    LinkedList<Boolean> list = new LinkedList<>();

    while (i > 0) {

        list.addFirst(isRightChild(i));

        i = parent(i);
    }
}
return list;
}
```

3 Graphs



- (a) Represent the above graph as an adjacency list and adjacency matrix

Depending on the convention being used, nodes may or may not have edges to themselves. For this problem, we stick with the convention that nodes do not have an edge to themselves.

A → [B, E, F]

B → [D]

C → []

D → [C, E]

E → []

F → [E]

	A	B	C	D	E	F
A	0	1	0	0	1	1
B	0	0	0	1	0	0
C	0	0	0	0	0	0
D	0	0	1	0	1	0
E	0	0	0	0	0	0
F	0	0	0	0	1	0

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.

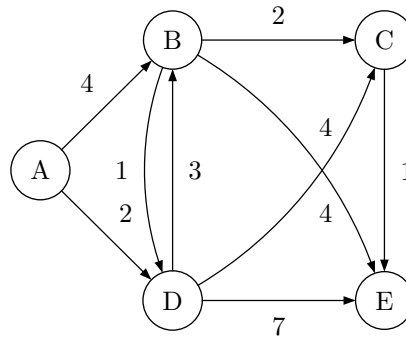
DFS has both a preorder and postorder traversal. Preorder means we visit the node before visiting its children. Postorder order means we visit the node only after visiting its children.

DFS preorder: A, B, D, C, E, F

DFS postorder: C, E, D, B, F, A

BFS: A, B, E, F, D, C

4 Dijkstra's Algorithm



- (a) Given the above graph, run Dijkstra's algorithm starting at node A. For each iteration, write down the entire state of the algorithm. This includes the value $\text{dist}(\mathbf{v})$ for all vertices $\mathbf{v}$ as well as what node was popped off of the fringe for that iteration.

*Note: If you want to keep track of the vertices traversed along the shortest paths from A to every other node in the graph, you will need to maintain an **edgeTo** array.*

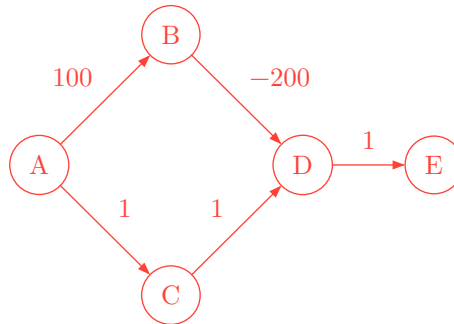
To run Dijkstra's algorithm, start with $\text{dist}(\mathbf{v})$ for all vertices $\mathbf{v}$ set to ∞ and a fringe that includes all the vertices. The fringe is a minimum priority queue that orders the vertices by $\text{dist}(\mathbf{v})$ values. At each iteration, pop off a node from the fringe (this will be the vertex in the fringe with the lowest $\text{dist}(\mathbf{v})$). For each outgoing edge e from this popped vertex, check to see whether the sum of $\text{dist}(\text{popped})$ and the edge e 's value is less than the current dist value of the vertex the edge connects to. If so, set the dist value of that vertex to this lower value. Note that vertices that have already been popped from the fringe will never have their dist values changed. This is because when we pop off a vertex, the distance to that vertex can only increase by considering other vertices and their edges (since the popped vertex currently has the minimum dist value). Continue until all nodes have been popped from the fringe.

v	Init	Pop A	Pop D	Pop B	Pop C	Pop E
A	0	0	0	0	0	0
B	∞	4	4	4	4	4
C	∞	∞	6	6	6	6
D	∞	2	2	2	2	2
E	∞	∞	9	8	7	7

*Note: For reference, here is the final **edgeTo** array that we get after running Dijkstra's algorithm to completion. $\text{edgeTo}(A)$ is "-" because it was used as the starting vertex. $\text{edgeTo}(C)$ is D because it is the first **edgeTo** vertex that was encountered along the shortest path to C. We would get D for $\text{edgeTo}(C)$ if we update the **edgeTo** and **dist** arrays when a candidate path is strictly shorter than the existing path. We would get B for $\text{edgeTo}(C)$ if we update the **edgeTo** and **dist** arrays when a candidate path is shorter than or the same distance as the existing path.*

- (b) What must be true about our graph in order to guarantee Dijkstra's will return the shortest path's tree to every vertex? Draw an example of a graph that demonstrates why Dijkstra's might fail if we do not satisfy this condition.

In order to guarantee Dijkstra's will return the shortest path to every vertex, we must have a graph that has no negative edge weights. Take the following graph as an example of why negative edge weights might cause an error:



For this graph, if we ran Dijkstra's starting from A, then we would get the incorrect shortest path to E since we would choose the bottom path through C instead of the top path through B.

We choose the bottom path because we reach and pop off vertices C, D, and E before popping off vertex B and considering its edge to D. This is because in Dijkstra's, when we pop off a vertex, we do so with the assumption that the distance to that vertex can only increase by considering other vertices and their edges (since the popped vertex currently has the min dist value). With negative edges, this assumption is no longer true.

Note that having negative edge weights does not guarantee Dijkstra's will fail, but if we have all non-negative edge weights then we are guaranteed to get the shortest path. When working with distances from the real world, we don't have to worry about negative edge weights because all distances in reality are strictly non-negative.

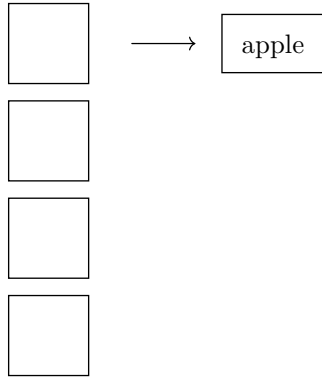
5 Hashing Practice

Given the provided `hashCode` implementation, hash the items listed below with separate chaining (the first item is already inserted for you). Assume the load factor is 1. Use geometric resizing with a resize factor of 2.

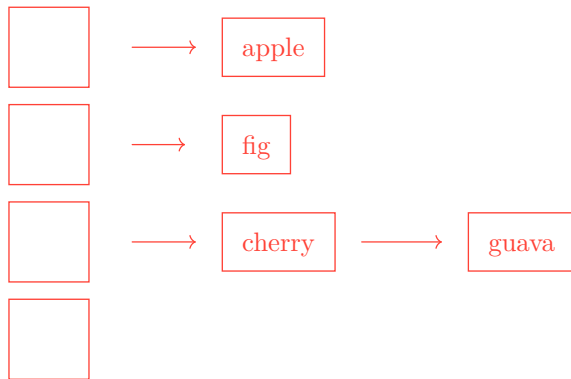
You may draw more boxes to extend the array when you need to resize.

```
/** Returns 0 if word begins with 'a', 1 if it begins with 'b', etc. */  
public int hashCode() {  
    return (int)(word.charAt(0) - 'a');  
}
```

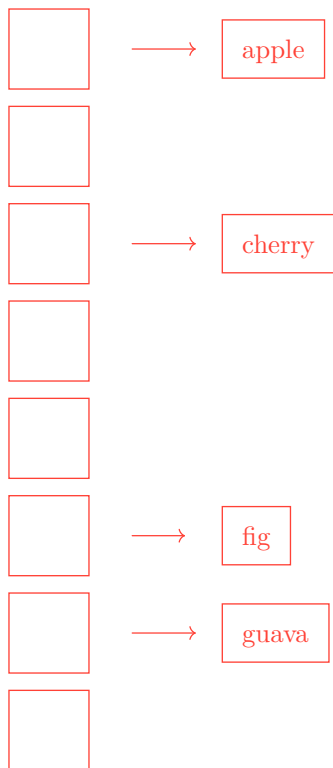
```
["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]
```



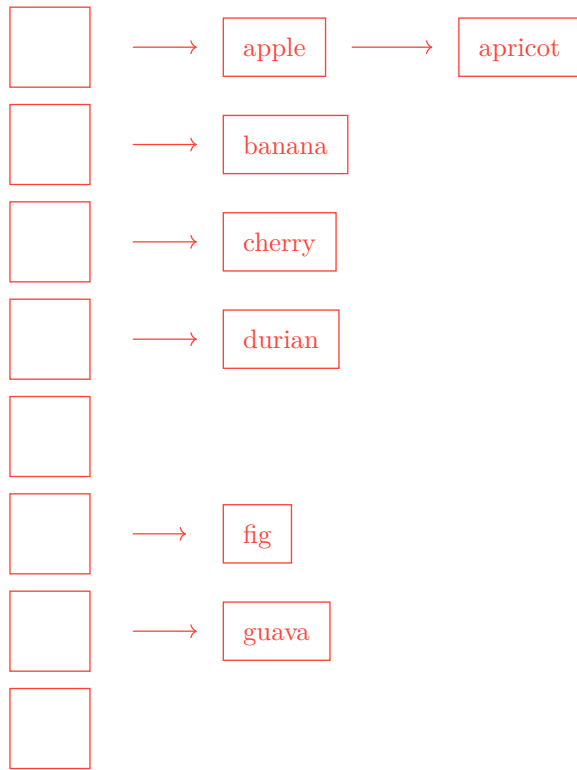
Here's what the hash table should look like after inserting **guava**:



However, at this point, our load factor is ≥ 1 , which means that we have to resize our hash table and re-distribute our elements. Here is what the hash table should look like after resizing:



Here is what the hash table should look like after all insertions have been completed:



6 LLRB-Chaining

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

- (a) Using this hash table,

The runtime of **contains** is, in the worst case:

Runtime: $\Theta (\log N)$

The runtime of **contains** is, in the best case:

Runtime: $\Theta (1)$

- (b) Evaluate the following statements as **true** or **false**, and briefly explain why.

“One advantage of using an LLRB for buckets is that it becomes possible to efficiently iterate over the keys in ascending order”

False. While it becomes possible to iterate over keys efficiently in ascending order within buckets using a tree traversal, it is still difficult to iterate over all of the hash table's keys, as each bucket contains a distinct LLRB.

“Assuming items in the hash table have good spread, we expect that an LLRB bucket would yield significantly better performance for **contains** and **add** than using an **ArrayList**”

False. If the items have good spread, number of elements in each bucket should be constant relative to N, so an asymptotic speedup doesn't yield significantly better performance.

7 Spread and Validity

Suppose we have the Dog class below.

```
public class Dog {
    private static int dogsCreated = 0;
    private int age;
    private final int microchipID;
    public String name;
    public String owner;

    public Dog(int age, String name, String owner) {
        Dog.dogsCreated += 1;
        this.age = age;
        this.name = name;
        this.owner = owner;
        this.microchipID = dogsCreated;
    }

    public void birthday() {
        this.age += 1;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Dog)) {
            return false;
        }
        Dog other = (Dog) o;
        return this.name.equals(other.name) && this.microchipID == other.microchipID;
    }

    @Override
    public int hashCode() {
        // Implementation omitted, see part (a)
    }

    ...
}
```

- (a) For each of the following implementations of the Dog's hashCode method, determine if it is **valid with good spread**, **valid with bad spread**, **valid with no spread**, or **invalid**.

Recall that “validity” depends on the following rules:

- Two objects that are equal (by the **equals** method) must have the same hash code
- An object's hash code only changes if its equality changes

```
@Override
public int hashCode() {
    return this.age;
}
```

Invalid - Dog equality does not depend on age, and a Dog's age is mutable

```
@Override
public int hashCode() {
    return this.microchipID;
}
```

Valid with Good Spread - microchipID is immutable, and has good spread.

```
@Override
public int hashCode() {
    return (int) this.name.charAt(0);
}
```

Valid with Bad Spread - **this.name** is immutable, but taking the first character as an integer has bad spread

```
@Override
public int hashCode() {
    return this.owner.hashCode();
}
```

Invalid - Two dogs that are not equal may have the same owner.

- (b) If a HashMap contains two or more items with equal keys, how might have this happened? Give one reason. Assume that we are referring to the built in java HashMap. Note that this part is independent from the previous; i.e. the keys aren't necessarily Dogs.

Either of these two cases are possible:

- The hashCode method of the item's class is non deterministic.
- One of the items was mutated after insertion, so now they are equal.